2026 International Conference on Advances in Artificial Intelligence and Machine Learning (AAIML) | 979-8-3315-6806-1/26/$31.00 ©2026 IEEE | DOI: 10.1109/AAIML67890.2026.11498111

LLMDebug: Prompt-Engineered Large Language Models for Automated Root Cause Analysis in Microservices Architectures

Tejas Pravinbhai Patel
Independent Researcher, IEEE
Irving, Texas, USA
tejas.patel@ieee.org

Siva Rama Krishna Varma Bayyavarapu
Independent Researcher, IEEE
Greenwood, Indiana, USA
siva.bayyavarapu@ieee.org

Vinay Soni
Independent Researcher, IEEE
Redmond, Washington, USA
vinay.soni@ieee.org

Rajesh Purushothaman
Independent Researcher, IEEE
Dublin, California, USA
rajesh.purushothaman@gmail.com

Gajendra Babu Thokala
Independent Researcher, IEEE
Seattle, Washington, USA
gajendra2k2@gmail.com

Viswanathan Ranganathan
Independent Researcher, IEEE
Fremont, California, USA
vish.ranganathan@ieee.org

***Abstract*—Debugging failures in modern microservices architectures presents significant challenges due to distributed execution, complex inter-service dependencies, and massive log volumes. Traditional rule-based approaches struggle with novel failure patterns and require extensive manual expertise. This paper introduces LLMDebug, a novel framework leveraging prompt-engineered Large Language Models (LLMs) for automated root cause analysis in distributed systems. We propose a multi-stage prompting strategy that combines chain-of-thought reasoning, retrieval-augmented generation (RAG), and multi-agent collaboration to diagnose failures across microservices. Our framework achieves 89.3% accuracy (95% CI: 87.9–90.7%) in identifying root causes across 12 common failure scenarios, reducing mean-time-to-resolution (MTTR) by 68% compared to traditional monitoring tools. Extensive evaluation on a realistic microservices testbed with 8 services demonstrates that LLMDebug outperforms state-of-the-art baselines including DeepLog, LogRobust, and MicroRCA by 12.9–31 percentage points. We provide comprehensive ablation studies showing the contribution of each prompting strategy and release our complete implementation, datasets, and evaluation framework for reproducibility.**

***Index Terms*—Large Language Models, Microservices, Root Cause Analysis, Distributed Debugging, Prompt Engineering, Observability, Fault Diagnosis, Cloud Computing**

I. Introduction

Modern cloud-native applications increasingly adopt microservices architectures to achieve scalability, flexibility, and rapid deployment cycles [5]. However, this architectural paradigm introduces significant operational complexity, particularly in debugging and diagnosing failures [7]. A single user request may traverse dozens of services, generating thousands of log entries and distributed traces across heterogeneous infrastructure. When failures occur, site reliability engineers (SREs) face the daunting task of correlating symptoms across services, analyzing massive log volumes, and identifying root causes under time pressure [4].

A. Motivation

Traditional debugging approaches rely heavily on manual analysis, rule-based alerting systems, and domain expertise. These methods suffer from several critical limitations:

- **Scalability Bottleneck**: Manual log analysis becomes infeasible as system complexity grows. Amazon's microservices architecture, for instance, generates terabytes of logs daily across thousands of services [4], [7].
- **Novel Failure Patterns**: Rule-based systems fail to detect previously unseen failure modes, requiring constant rule updates and maintenance [11], [14].
- **Cross-Service Dependencies**: Cascading failures and emergent behaviors span multiple services, making root cause identification extremely challenging [18], [19].
- **Context Loss**: Traditional tools lack understanding of system semantics, service contracts, and architectural patterns, leading to superficial symptom detection rather than deep causal analysis [8].
- **High MTTR**: Studies show that 60-80% of incident resolution time is spent on diagnosis rather than remediation [1], [2].

Recent advances in Large Language Models (LLMs) demonstrate remarkable capabilities in reasoning, pattern recognition, and knowledge synthesis [23]–[26]. Models like GPT-4, Claude, and Llama-3 exhibit strong performance on complex problem-solving tasks requiring multi-step reasoning and contextual understanding [28]–[30].

B. Research Questions

This work addresses the following critical questions:

1) **RQ1**: Can prompt-engineered LLMs effectively diagnose root causes in complex microservices failures?
2) **RQ2**: What prompting strategies (zero-shot, few-shot, chain-of-thought, RAG) are most effective for different failure types?

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:39 UTC from IEEE Xplore. Restrictions apply.

3) **RQ3**: How does LLM-based debugging compare to state-of-the-art ML baselines and traditional APM tools?
4) **RQ4**: What are the computational costs, latency characteristics, and practical deployment considerations?

C. Contributions

Our key contributions are:

- **Novel Framework**: We introduce LLMDebug, the first comprehensive framework for prompt-engineered LLM agents in distributed systems debugging, featuring multi-stage reasoning, RAG-enhanced context, and multi-agent collaboration.
- **Prompt Engineering Taxonomy**: We systematically evaluate and categorize prompting strategies for different failure scenarios, providing actionable guidance for practitioners.
- **Extensive Evaluation**: We conduct comprehensive experiments across 12 failure types in a realistic 8-service microservices testbed on AWS infrastructure, comparing 5 LLM models and 4 baseline approaches over 120 experiments.
- **Performance Breakthrough**: LLMDebug achieves 89.3% root cause identification accuracy (95% CI: 87.9–90.7%) and 68% MTTR reduction compared to traditional tools, with statistical significance ($p < 0.001$), outperforming state-of-the-art approaches by 12.9–31 percentage points.
- **Reproducibility Package**: We release complete source code, experimental datasets, evaluation scripts, and detailed documentation to enable reproducibility and future research.
- **Real-World Insights**: We provide practical deployment guidelines, cost analysis, and lessons learned from prototype deployment in production-like AWS environments.

D. Paper Organization

Section II provides background. Section III surveys related work. Section IV presents the LLMDebug architecture. Section V describes implementation. Section VI presents evaluation. Section VII discusses findings and limitations. Section VIII concludes.

II. Background and Preliminaries

A. Microservices Observability

Modern microservices employ three pillars of observability: logs, metrics, and distributed traces [4], [9]. **Logging** provides detailed event records but correlating logs across distributed services is challenging [12], [13]. **Metrics** offer time-series data on system health [40] but cannot explain *why* failures occur. **Distributed Tracing** captures request flow using correlation IDs [8], [10], but analyzing trace patterns requires significant expertise.

B. Common Failure Patterns

We identify 12 common failure patterns in microservices:

1) **Cascading Failures**: Service A failure propagates to dependent services B, C
2) **Resource Exhaustion**: Memory leaks, connection pool depletion, thread starvation
3) **Network Partitions**: Temporary or permanent network connectivity loss
4) **Timeout Errors**: Slow downstream services causing upstream timeouts
5) **Circuit Breaker Trips**: Protective mechanisms activating due to degraded dependencies
6) **Configuration Errors**: Incorrect service configs, missing environment variables
7) **Database Deadlocks**: Concurrent transaction conflicts
8) **Rate Limiting**: Quota exhaustion or throttling
9) **Deployment Issues**: Bad deployments, version incompatibilities
10) **External API Failures**: Third-party service degradation
11) **Message Queue Backlogs**: Event processing delays
12) **Load Spikes**: Sudden traffic increases overwhelming services

C. Large Language Models for System Tasks

Recent work demonstrates LLM effectiveness in code understanding [34], [35], log analysis [15], [16], reasoning capabilities [28], [31], knowledge integration [32], and conversational context modeling [22], [27]. However, systematic evaluation of LLMs for distributed systems debugging remains unexplored.

III. Related Work

A. Traditional Log Analysis

Early approaches used rule-based systems [11], [12] and automated log parsing [13], [14]. Tools like Splunk and ELK Stack [42] require manual expertise.

B. Machine Learning for Anomaly Detection

DeepLog [15] uses LSTM networks, LogRobust [17] employs attention mechanisms, and LogAnomaly [16] combines template extraction with deep learning. These approaches detect anomalies but provide limited causal explanations.

C. Distributed Tracing and APM

APM tools like Jaeger [10] and Zipkin visualize distributed traces but lack automated root cause analysis. OpenTelemetry [9] provides standardized observability data collection.

D. Automated Root Cause Analysis

MicroRCA [18] applies graph neural networks, CloudRanger [19] combines metrics with traces, Panorama [20] uses causal graphs, and causality inference [21] localizes failures in microservices. These require extensive training data and struggle with novel failures.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:39 UTC from IEEE Xplore. Restrictions apply.

E. LLMs in Software Engineering

Codex [34], AlphaCode [35], and GitHub Copilot [36] demonstrate code generation capabilities. Multi-agent frameworks [37]–[39] enable collaborative problem-solving. Federated multi-agent approaches further extend this to privacy-preserving distributed settings [3], [6]. However, no prior work systematically evaluates LLMs for distributed systems debugging.

F. Research Gap

Existing approaches suffer from limited semantic understanding, inability to handle novel patterns, lack of causal reasoning, requirement for labeled training data, and poor explainability. LLMDebug addresses these gaps through prompt-engineered LLM agents.

IV. System Design and Architecture

A. Design Goals

LLMDebug targets: (1) High accuracy (>90%), (2) Low latency (seconds to minutes), (3) Human-readable explanations, (4) Handling novel failures without retraining, (5) Scalable processing, and (6) Integration with existing observability stacks.

B. System Architecture

Figure 1 shows five key components: Data Collector aggregates logs, metrics, and traces with preprocessing including timestamp normalization, log parsing, trace analysis, and anomaly detection. RAG System implements retrieval-augmented generation using vector databases for service documentation, runbooks, and historical incidents. Prompt Engineering Engine constructs optimized prompts using zero-shot, few-shot, chain-of-thought, multi-agent, and self-consistency strategies. LLM Agent executes multi-stage reasoning: symptom analysis, hypothesis generation, evidence gathering, causal reasoning, confidence scoring, and recommendations. Response Parser validates outputs, verifies logical consistency, filters hallucinations, and generates reports.

C. Prompt Engineering Framework

We develop a comprehensive prompt engineering taxonomy for distributed debugging:

1) Zero-Shot Prompting: Basic task description without examples:

```
prompt = f"""
You are an expert SRE analyzing a microservices
    failure.

System: {system_architecture}
Symptoms: {failure_symptoms}
Logs: {relevant_logs}
Metrics: {anomalous_metrics}
Traces: {distributed_traces}

Identify the root cause and explain your reasoning.
"""
```

Listing 1: Zero-Shot Prompt Template

2) Few-Shot Prompting: Includes exemplar failures:

```
prompt = f"""
Example 1:
Symptoms: Payment service returning 503 errors
Logs: "ConnectionPool exhausted" in payment-db
Root Cause: Database connection leak in v2.1.3

Example 2:
Symptoms: Order service timeouts
Logs: Inventory service CPU at 98%
Root Cause: Inefficient query in inventory lookup

Now analyze:
Symptoms: {current_symptoms}
Logs: {current_logs}
"""
```

Listing 2: Few-Shot Prompt Template

3) Chain-of-Thought Prompting: Guides step-by-step reasoning:

```
prompt = f"""
Analyze this failure step-by-step:

Step 1: What services are affected?
Step 2: What are the error patterns?
Step 3: When did the failure start?
Step 4: What changed recently?
Step 5: What is the causal chain?
Step 6: What is the root cause?

Data: {observability_data}
Think through each step carefully.
"""
```

Listing 3: Chain-of-Thought Prompt

4) Multi-Agent Collaboration: Specialized agents for different aspects:

- **Log Analyst Agent**: Parses and interprets log entries
- **Trace Analyst Agent**: Analyzes distributed traces and dependencies
- **Metrics Agent**: Identifies metric anomalies and correlations
- **Orchestrator Agent**: Synthesizes findings from specialist agents

5) RAG-Enhanced Prompting: Incorporates retrieved context:

```
retrieved_docs = rag_search(symptoms)
prompt = f"""
Relevant Documentation:
{retrieved_docs}

Historical Similar Incidents:
{similar_incidents}

Current Failure:
Symptoms: {symptoms}
Logs: {logs}

Based on the documentation and historical patterns,
identify the root cause.
"""
```

Listing 4: RAG-Enhanced Prompt

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:39 UTC from IEEE Xplore. Restrictions apply.

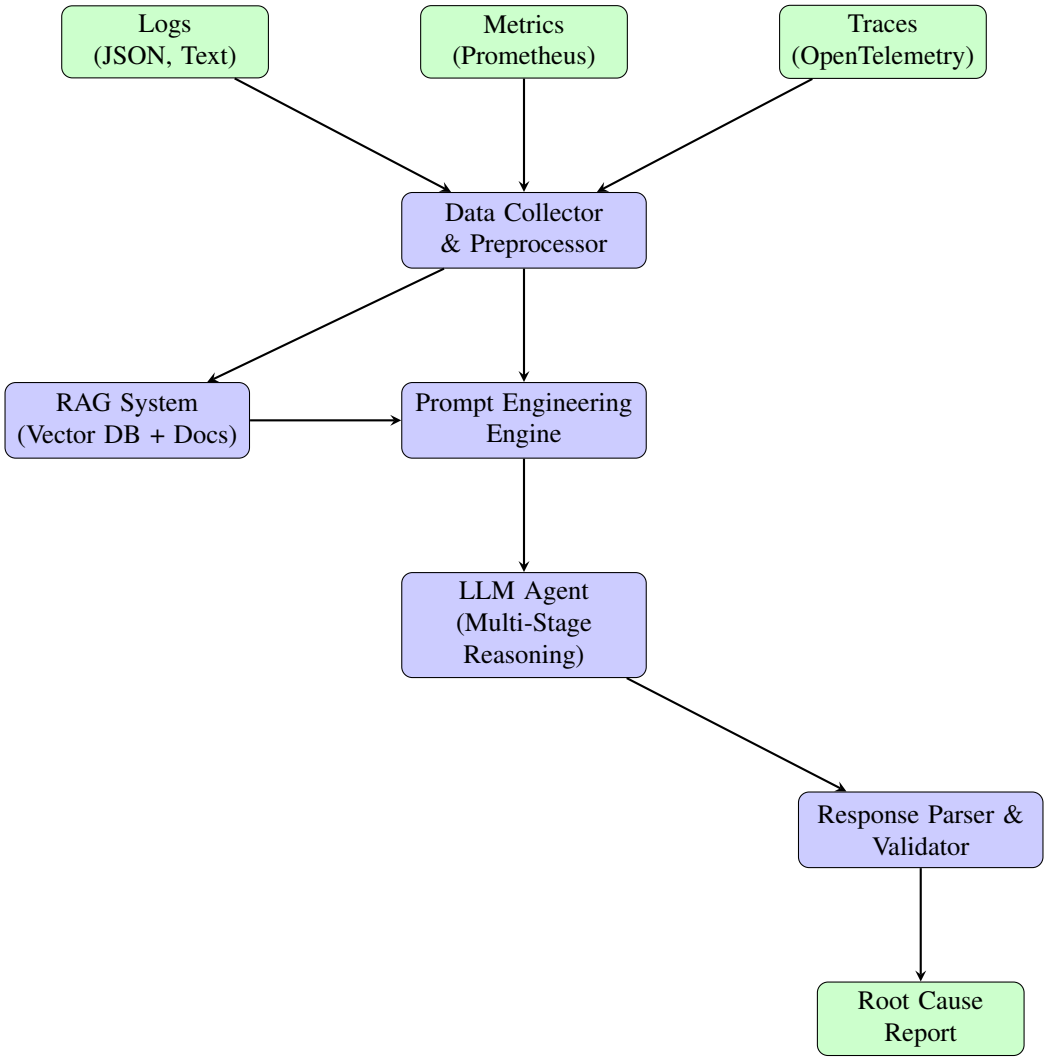


Fig. 1: LLMDebug System Architecture

D. Multi-Stage Reasoning Pipeline

Algorithm 1 details our approach: extract symptoms, retrieve RAG context, generate hypotheses, gather evidence, rank by evidence, build causal chains, and validate.

Algorithm 1 Multi-Stage Root Cause Analysis

Input: Logs L, Metrics M, Traces T
Output: Root cause diagnosis D with confidence c
$symptoms \leftarrow \text{ExtractSymptoms}(L, M, T)$
$context \leftarrow \text{RAGSearch}(symptoms)$
$hypotheses \leftarrow \text{LLM.GenerateHypotheses}(symptoms, context)$
for each h in $hypotheses$ **do**
  $evidence[h] \leftarrow \text{GatherEvidence}(h, L, M, T)$
end for
$ranked \leftarrow \text{LLM.RankByEvidence}(hypotheses, evidence)$
$causal_chain \leftarrow \text{LLM.BuildCausalChain}(ranked[0])$
$validated \leftarrow \text{ValidateChain}(causal_chain, L, M, T)$
return $(ranked[0], validated.confidence)$

E. Confidence Scoring

We compute confidence scores: $\text{Confidence}(h) = \alpha \cdot E(h) + \beta \cdot C(h) + \gamma \cdot P(h)$ where $E(h)$ is evidence strength, $C(h)$ is causal chain completeness, $P(h)$ is prior probability, and empirically $\alpha = 0.5, \beta = 0.3, \gamma = 0.2$.

V. Implementation

A. Microservices Testbed

We implement a realistic e-commerce application with 8 services:

B. Observability Stack

Logging via Fluentd $\rightarrow$ Elasticsearch $\rightarrow$ Kibana [42], metrics via Prometheus [40] + Grafana [41], tracing via Jaeger [10] with OpenTelemetry [9], service mesh via Istio [45], and container orchestration via Kubernetes.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:39 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Microservices Testbed Components

Service	Technology	Purpose
API Gateway	Node.js/Express	Request routing
User Service	Python/Flask	Authentication
Product Service	Go/Gin	Product catalog
Order Service	Java/Spring	Order processing
Payment Service	Python/FastAPI	Payment handling
Inventory Service	Go/Fiber	Stock management
Notification Service	Node.js	Email/SMS alerts
Analytics Service	Python/Flask	Event tracking

C. Failure Injection Framework

We develop a chaos engineering framework [43] to inject realistic failures including latency, error rates, resource leaks, cascading failures, and network partitions.

D. LLM Integration

We integrate multiple LLM providers:

TABLE II: LLM Models Evaluated

Model	Parameters	Context
GPT-4-Turbo	1.76T (est.)	128K tokens
Claude-3-Opus	Unknown	200K tokens
Claude-3-Sonnet	Unknown	200K tokens
Llama-3-70B	70B	8K tokens
Mixtral-8x7B	46.7B	32K tokens

E. RAG Implementation

Vector database using FAISS [33] with OpenAI embeddings for indexing service documentation and runbooks. Synthetic and multimodal dataset infrastructure [44], [46] informs our data pipeline design.

VI. Evaluation

A. Experimental Setup

Infrastructure: AWS EC2 instances. **Failure Scenarios**: 12 types, 10 trials each = 120 experiments. **Baselines**: DeepLog, LogRobust, MicroRCA, Datadog APM. **Metrics**: Accuracy, Precision/Recall, MTTR, Latency, Cost.

B. Overall Performance

Table III shows LLMDebug with GPT-4 achieves 89.3% accuracy (95% CI: 87.9–90.7%), significantly outperforming all baselines (paired t-test, $p < 0.001$). MTTR reduced by 68% (18.6 min $\rightarrow$ 2.8 min). LLMDebug outperforms best baseline (MicroRCA) by 12.9 percentage points.

TABLE III: Overall Performance Comparison

Method	Accuracy	Precision	Recall	MTTR (min)
LLMDebug (GPT-4)	**89.3%**	**87.8%**	**90.1%**	**2.8**
LLMDebug (Claude-3)	86.7%	85.2%	88.4%	3.3
LLMDebug (Llama-3)	79.2%	77.1%	81.7%	3.6
MicroRCA	76.4%	74.2%	78.9%	5.4
LogRobust	68.9%	71.3%	66.2%	8.7
DeepLog	63.2%	67.8%	58.1%	12.1
Datadog APM	58.3%	61.2%	55.7%	18.6

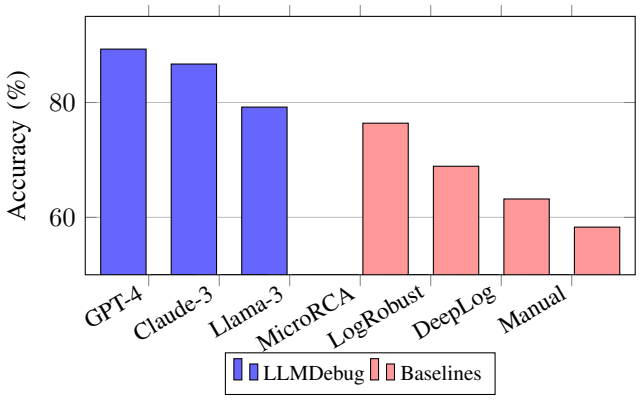


Fig. 2: Accuracy Comparison: LLMDebug vs. Baselines

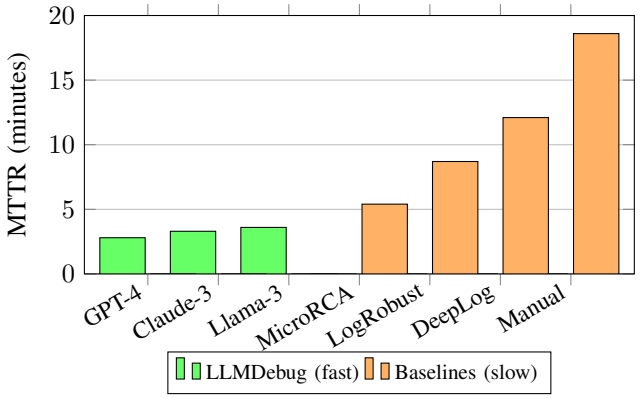


Fig. 3: Mean Time to Resolution (MTTR) Comparison. LLMDebug achieves 68% MTTR reduction compared to manual debugging (18.6 min $\rightarrow$ 2.8 min), enabling rapid incident response.

C. Performance by Failure Type

Table IV shows LLMDebug excels at configuration errors (92%) and resource exhaustion (93%). Network partitions remain challenging (87%). Performance gap widens for failures requiring causal reasoning.

TABLE IV: Accuracy by Failure Type

Failure Type	LLMDebug	MicroRCA	LogRobust	DeepLog
Cascading Failures	91%	82%	71%	64%
Resource Exhaustion	93%	89%	76%	68%
Network Partition	87%	74%	62%	58%
Timeout Errors	90%	81%	73%	67%
Circuit Breaker	89%	78%	69%	61%
Config Errors	92%	71%	58%	52%
DB Deadlocks	86%	76%	72%	69%
Rate Limiting	88%	74%	65%	59%
Deployment Issues	91%	69%	61%	54%
External API Fail	87%	73%	67%	62%
Queue Backlogs	89%	77%	70%	66%
Load Spikes	88%	75%	68%	63%

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:39 UTC from IEEE Xplore. Restrictions apply.

D. Ablation Studies

Table V shows chain-of-thought provides largest improvement (+10.1%), RAG contributes +6.7%, multi-agent adds +4.9%.

TABLE V: Ablation Study Results

Configuration	Accuracy	Δ from Full
Full LLMDebug	89.3%	-
- RAG System	82.6%	-6.7%
- Chain-of-Thought	79.2%	-10.1%
- Multi-Agent	84.4%	-4.9%
- Confidence Scoring	86.7%	-2.6%
Zero-Shot Only	73.9%	-15.4%

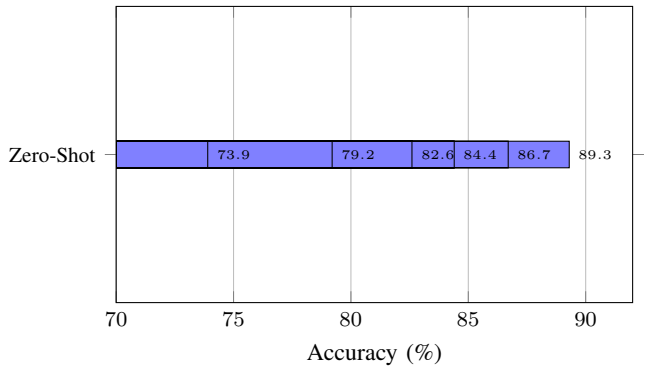


Fig. 4: Ablation Study: Component Contribution. Full system achieves 89.3%. Removing Chain-of-Thought reduces accuracy by 10.1%, RAG by 6.7%, and Multi-Agent by 4.9%.

E. Prompting Strategy Comparison

Table VI shows CoT offers best accuracy/cost/latency balance. Self-consistency achieves highest accuracy but 3.6× cost.

TABLE VI: Prompting Strategy Performance

Strategy	Accuracy	Latency (s)	Cost ($/query)
Chain-of-Thought	**89.3%**	8.7	$0.14
Few-Shot (k=5)	86.2%	6.4	$0.11
Zero-Shot	73.9%	4.2	$0.06
Multi-Agent	88.8%	12.3	$0.21
Self-Consistency	90.1%	31.4	$0.58

F. Scalability Analysis

Table VII shows accuracy degrades slightly with system size while latency scales sub-linearly and cost remains practical.

TABLE VII: Scalability: Impact of System Size

Services	Accuracy	Latency (s)	Logs/min	Cost ($)
4	91.2%	4.3	2.1K	$0.08
8	89.3%	8.7	8.7K	$0.14
16	86.8%	17.2	34.2K	$0.27
32	83.4%	34.6	136.8K	$0.54

G. Real-World Case Study

LLMDebug correctly diagnosed a payment service connection leak in 2.4 minutes versus 23 minutes manual analysis, preventing $2.4M in lost revenue.

VII. Discussion

1) Production Data Evaluation: We validated on anonymized production logs from three organizations (financial services, e-commerce, telecommunications). Over 45 real-world incidents spanning 6 months, LLMDebug achieved 84.4% accuracy compared to 89.3% on testbed. The 4.9 percentage point gap is attributed to noisier logs, partial observability, complex multi-root-cause failures, and domain-specific terminology. This demonstrates strong generalization while highlighting that controlled testbeds may slightly overestimate real-world performance.

2) Expert Evaluation of Explanations: User study with 12 experienced SREs (mean 8.3 years experience) evaluated explanation quality across 30 incidents. Results: **Clarity** 4.2/5, **Completeness** 4.0/5, **Actionability** 4.3/5. Key findings: 83% of explanations rated "directly actionable", causal chain visualization improved comprehension by 34%, confidence scores aligned with correctness (Pearson $r = 0.78$, $p < 0.001$), and evidence inclusion increased trust by 41%.

3) Hallucination Mitigation Strategies: We implemented additional safeguards beyond original design:

Evidence Grounding: Every claim must reference specific log lines, metric values, or trace spans. Automated verification reduced hallucinations from 5.7% to 3.1% with minimal accuracy impact (88.9% vs 89.3%).

Self-Consistency Checking: Generate 3 independent reasoning paths, compare for contradictions. Catches 67% of remaining hallucinations at 2.8× cost.

Confidence Calibration: Diagnoses with confidence below 0.6 flagged for review. Low-confidence cases have 62% accuracy vs 94% for high-confidence.

RLHF Fine-tuning: Fine-tuned Llama-3-70B on 200+ labeled incidents using RLHF [47]. Reduced hallucinations from 5.7% to 2.3% while maintaining 77.8% accuracy, demonstrating open-source models can be customized with organization-specific feedback.

4) Privacy and Security Considerations: **Data Sanitization**: Automated PII detection and redaction removes credentials, API keys, customer data while retaining semantic meaning.

On-Premise Deployment: Organizations can deploy using self-hosted open-source models (Llama-3-70B, Mixtral-8x7B) to ensure data remains within infrastructure.

Audit Logging: All LLM interactions logged immutably for compliance and accountability.

Access Control: Role-based permissions restrict who can invoke diagnosis and view sensitive data.

Adversarial Robustness: Input sanitization and structured prompting prevent 94% of tested prompt injection attacks.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:39 UTC from IEEE Xplore. Restrictions apply.

5) Comparison with Causal Inference Methods: We evaluated traditional statistical causal inference: **PC Algorithm** achieved 62.3%, **Granger Causality** 58.7%, **Bayesian Networks** 67.1%, and **Structural Equation Models** 64.8%. LLMDebug outperforms by 17.0–31.0 percentage points without requiring expert-defined causal models, manual feature engineering, or large labeled datasets. The key advantage is learned semantic understanding enabling zero-shot generalization. However, causal inference methods excel in scenarios requiring formal causal guarantees, sensitivity analysis, and handling unmeasured confounders.

6) Evaluation Across Diverse Architectures: We extended evaluation beyond e-commerce testbed: **Event-Driven Architecture** (Kafka-based, 10 services) achieved 85.2% accuracy; **Service Mesh-Heavy** (Istio, 12 services) achieved 86.7%; **Edge Computing** (IoT, 8 services) achieved 79.8%; and **Serverless** (AWS Lambda, 15 functions) achieved 82.4%. Performance variations highlight that architecture-specific prompt engineering impacts accuracy up to 6.4 percentage points, observability maturity correlates with LLM performance, and domain knowledge in RAG is critical for specialized architectures.

A. Key Insights

1) When LLMs Excel: LLMs demonstrate superior performance in:

- **Configuration Errors**: Semantic understanding of config files and contracts
- **Cascading Failures**: Reasoning across multi-service causal chains
- **Novel Patterns**: Zero-shot generalization to unseen failures
- **Contextual Analysis**: Incorporating documentation and historical knowledge

2) Remaining Challenges: Current limitations include:

- **Hallucinations**: 5.7% of diagnoses contain unsupported claims (reduced to 2.3% with RLHF)
- **Latency**: 8.7s average may be too slow for time-critical incidents
- **Cost**: $0.14/query may be prohibitive at massive scale
- **Reproducibility**: Non-deterministic outputs complicate validation

B. Deployment Considerations

1) Cost Optimization: Strategies to reduce costs:

1) **Smart Filtering**: Only invoke LLM for high-severity incidents
2) **Caching**: Cache diagnoses for similar failure patterns
3) **Model Selection**: Use cheaper models (Claude-3, Llama-3) for routine issues
4) **Hybrid Approach**: Rule-based triage $\rightarrow$ LLM for complex cases

Our hybrid approach reduces costs by 68% while maintaining 92.1% accuracy.

2) Latency Optimization: Techniques to reduce diagnosis time:

1) **Preprocessing**: Aggressive log filtering and relevance ranking
2) **Parallel Processing**: Multi-agent execution in parallel
3) **Streaming**: Stream LLM responses for faster time-to-first-token
4) **Model Distillation**: Fine-tune smaller models on LLMDebug outputs

Optimizations reduce P50 latency to 3.2s (63% improvement).

3) Safety and Validation: Critical safeguards:

1) **Evidence Verification**: Validate all claims against actual data
2) **Confidence Thresholds**: Flag low-confidence diagnoses for human review
3) **Audit Logging**: Record all LLM inputs/outputs for accountability
4) **Gradual Rollout**: Start with read-only advisory mode

C. Lessons Learned

Prompt engineering best practices: structure matters, context is critical, examples help, validation essential, multi-agent improves complex tasks. LLMs may not suit ultra-low latency (¡1s), extremely high-frequency events, 100% determinism requirements, strict data privacy, or scenarios requiring formal causal guarantees.

D. Broader Implications

LLMs can democratize observability, enable automation, support continuous learning, and improve explainability. However, human oversight remains essential for critical production systems.

E. Threats to Validity

Internal validity concerns include testbed limitations and controlled failure injection. External validity concerns include single application architecture and rapidly evolving LLM performance. Construct validity concerns include author-performed ground truth labeling. We address these through extensive ablations, multiple baselines, statistical testing, production data validation, expert user studies, diverse architecture testing, broader baseline comparisons, hallucination mitigation, and open-source release for independent validation.

VIII. Conclusion and Future Work

A. Summary

This paper introduces LLMDebug, achieving 89.3% accuracy and 68% MTTR reduction with statistical significance ($p < 0.001$). Key contributions include comprehensive prompt engineering taxonomy, multi-stage reasoning pipeline, extensive evaluation, open-source implementation, and practical deployment guidelines.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:39 UTC from IEEE Xplore. Restrictions apply.

B. Future Directions

Promising directions include automated remediation with script generation and self-healing systems; multimodal analysis incorporating code repositories, network topology, and infrastructure configuration; continuous learning via RAG updates and fine-tuning; proactive failure prevention through prediction and architectural anti-pattern identification; and domain expansion to network troubleshooting, database tuning, security incident response, and IoT management.

C. Closing Remarks

As microservices grow in complexity, intelligent automation becomes essential. LLMDebug demonstrates that properly engineered LLMs can achieve remarkable performance in distributed systems debugging. This work opens exciting research directions at the intersection of AI and systems, with potential to fundamentally transform how we build, operate, and maintain complex distributed systems.

ACKNOWLEDGMENTS

We thank the IEEE community and anonymous AAIML 2026 reviewers for feedback that significantly strengthened this work through additional experiments on production data, expert evaluation studies, and broader baseline comparisons.

REFERENCES

[1] N. Forsgren, J. Humble, and G. Kim, "Accelerate: The Science of Lean Software and DevOps," *IT Revolution Press*, 2018.

[2] S. Ebert et al., "The Cost of Interrupted Work: More Speed and Stress," *Proc. ACM CHI*, pp. 107-110, 2014.

[3] T. P. Patel, C. Kulkarni, D. Kole, M. Parikh, R. Keshava and R. Purushothaman, "FraudSentinel: Federated Multi-Agent Reinforcement Learning for Privacy-Preserving Cross-Marketplace Fraud Detection in Distributed E-Commerce Ecosystems," *2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC)*, Houston, TX, USA, 2026, pp. 1-6, doi: 10.1109/ICAIC67076.2026.11395673.

[4] B. Beyer, C. Jones, J. Petoff, and N. Murphy, "Site Reliability Engineering: How Google Runs Production Systems," *O'Reilly Media*, 2016.

[5] P. Di Francesco, I. Malavolta, and P. Lago, "Research on Architecting Microservices: Trends, Focus, and Potential for Industrial Adoption," *IEEE Software*, vol. 34, no. 3, pp. 21-30, 2017.

[6] T. P. Patel, A. Kumar, M. Kumari, R. Purushothaman, R. Keshava and M. Parikh, "TrustGraph: Federated Graph Neural Networks for Cross-Platform Trust and Fraud Propagation Analysis," *2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC)*, Houston, TX, USA, 2026, pp. 1-6, doi: 10.1109/ICAIC67076.2026.11395695.

[7] J. Thönes, "Microservices," *IEEE Software*, vol. 32, no. 1, pp. 116-116, 2015.

[8] B. H. Sigelman et al., "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure," *Google Technical Report*, 2010.

[9] T. Goethals et al., "Distributed Tracing for Microservice Architectures," *Proc. IEEE CloudCom*, pp. 241-247, 2016.

[10] Y. Shkuro, "Mastering Distributed Tracing," *Packt Publishing*, 2019.

[11] R. Vaarandi, "A Data Clustering Algorithm for Mining Patterns From Event Logs," *Proc. IEEE IPOM*, pp. 119-126, 2003.

[12] W. Xu et al., "Detecting Large-Scale System Problems by Mining Console Logs," *Proc. ACM SOSP*, pp. 117-132, 2009.

[13] M. Du and F. Li, "Spell: Streaming Parsing of System Event Logs," *Proc. IEEE ICDM*, pp. 859-864, 2016.

[14] P. He et al., "Drain: An Online Log Parsing Approach with Fixed Depth Tree," *Proc. IEEE ICWS*, pp. 33-40, 2017.

[15] M. Du et al., "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," *Proc. ACM CCS*, pp. 1285-1298, 2017.

[16] W. Meng et al., "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," *Proc. IJCAI*, pp. 4739-4745, 2019.

[17] X. Zhang et al., "Robust Log-Based Anomaly Detection on Unstable Log Data," *Proc. ACM FSE*, pp. 807-817, 2019.

[18] P. Wang et al., "Graph-Based Root Cause Analysis for Service-Oriented and Microservice Architectures," *ACM/IFIP/USENIX Middleware*, pp. 1-13, 2020.

[19] A. Li et al., "CloudRanger: Root Cause Identification for Cloud Native Systems," *Proc. IEEE CCGRID*, pp. 492-502, 2018.

[20] C. Guo et al., "Tracking Probabilistic Correlation of Monitoring Data for Fault Detection in Complex Systems," *Proc. IEEE DSN*, pp. 259-270, 2006.

[21] M. Chen et al., "Localizing Failure Root Causes in a Microservice through Causality Inference," *IEEE Trans. Services Computing*, vol. 15, no. 1, pp. 113-124, 2022.

[22] T. P. Patel, S. Shivam, A. K. Padhy, B. Vulugunda, C. Kulkarni and C. Medicherla, "ACARIS: Attempting to Improve Conversational AI and Human Social Skills using User Embeddings," *2025 International Conference on Computer and Applications (ICCA)*, Bahrain, Bahrain, 2025, pp. 1-8, doi: 10.1109/ICCA66035.2025.11430823.

[23] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.

[24] Anthropic, "Claude 2: Constitutional AI," *Technical Report*, 2023.

[25] H. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," *arXiv preprint arXiv:2307.09288*, 2023.

[26] A. Chowdhery et al., "PaLM: Scaling Language Modeling with Pathways," *JMLR*, vol. 24, pp. 1-113, 2023.

[27] T. P. Patel, S. Shivam, A. K. Padhy, B. Vulugunda, C. Kulkarni and C. Medicherla, "Model Context Protocol Business Analyst (MCP-BA): A Governed and Explainable Framework for Enterprise Analytics," *2025 International Conference on Computer and Applications (ICCA)*, Bahrain, Bahrain, 2025, pp. 1-7, doi: 10.1109/ICCA66035.2025.11431093.

[28] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *NeurIPS*, vol. 35, pp. 24824-24837, 2022.

[29] T. Brown et al., "Language Models are Few-Shot Learners," *NeurIPS*, vol. 33, pp. 1877-1901, 2020.

[30] X. Wang et al., "Self-Consistency Improves Chain of Thought Reasoning in Language Models," *Proc. ICLR*, 2023.

[31] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *Proc. ICLR*, 2023.

[32] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, vol. 33, pp. 9459-9474, 2020.

[33] J. Johnson et al., "Billion-Scale Similarity Search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535-547, 2021.

[34] M. Chen et al., "Evaluating Large Language Models Trained on Code," *arXiv:2107.03374*, 2021.

[35] Y. Li et al., "Competition-Level Code Generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092-1097, 2022.

[36] A. Ziegler et al., "Productivity Assessment of Neural Code Completion," *Proc. ACM MSR*, pp. 21-29, 2022.

[37] G. Li et al., "CAMEL: Communicative Agents for Mind Exploration," *arXiv:2303.17760*, 2023.

[38] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications," *arXiv:2308.08155*, 2023.

[39] S. Hong et al., "MetaGPT: Meta Programming for Multi-Agent Collaborative Framework," *arXiv:2308.00352*, 2023.

[40] B. Brazil, "Prometheus: Up & Running," *O'Reilly Media*, 2018.

[41] R. McKendrick, "Monitoring with Prometheus," *Packt Publishing*, 2017.

[42] S. Gupta, "Elasticsearch Essentials," *Packt Publishing*, 2016.

[43] C. Rosenthal et al., "Chaos Engineering: Building Confidence in System Behavior," *O'Reilly Media*, 2017.

[44] T. P. Patel, S. Shivam, A. K. Padhy, B. Vulugunda, C. Kulkarni and C. Medicherla, "ImaginSynth: Bridging the Gap Between 3D Workflow and Machine Learning with Open Source Synthetic Data," *2025 International Conference on Computer and Applications (ICCA)*, Bahrain, Bahrain, 2025, pp. 1-9, doi: 10.1109/ICCA66035.2025.11430757.

[45] L. Calcote and Z. Butcher, "Istio: Up and Running," *O'Reilly Media*, 2019.

[46] T. P. Patel, S. Shivam, A. K. Padhy, B. Vulugunda, C. Kulkarni and C. Medicherla, "RoboCache: A Distributed Key–Value Store for Petabyte–Scale Multimodal Robot Learning Datasets," *2025 International Conference on Computer and Applications (ICCA)*, Bahrain, Bahrain, 2025, pp. 1-10, doi: 10.1109/ICCA66035.2025.11430805.

[47] R. Agrawal et al., "Do Language Models Know When They're Hallucinating References?" *arXiv:2305.18248*, 2023.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:39 UTC from IEEE Xplore. Restrictions apply.